PYTHON UNIT 3 QB

(4 - marks)

1. Define exception handling in Python and explain its importance.

1. Exception handling in Python is a mechanism used to handle runtime errors that occur during program execution. It helps prevent the program from crashing and allows the program to continue running normally.
2. In Python, errors that occur after the syntax test are called exceptions or logical errors. These errors happen during execution, such as dividing by zero or opening a file that does not exist.
3. Exception handling uses keywords like try, except, else, and finally to detect and manage errors in a program. The risky code is written in the try block and the error-handling code is written in the except block.
4. The importance of exception handling is that it improves program reliability and makes debugging easier. It also separates normal program logic from error-handling code and helps manage unexpected situations effectively.

2. Differentiate between syntax errors and exceptions with examples.

1. Syntax errors occur when the Python interpreter cannot understand the code because it violates the rules of the language. Exceptions occur during program execution even when the syntax of the program is correct.
2. A syntax error stops the program from running completely because the interpreter cannot process the code. An exception occurs during runtime and can be handled using exception handling techniques.
3. An example of a syntax error is a missing parenthesis or colon in a Python statement. For example: print("Hello" will produce a syntax error because the closing parenthesis is missing.
4. An example of an exception is dividing a number by zero or opening a file that does not exist. For example: res = 10 / 0 produces a ZeroDivisionError during program execution.

3. Write a Python program to handle ZeroDivisionError using try-except.

1. A ZeroDivisionError occurs when a program tries to divide a number by zero during execution. This error can be handled using the try-except block in Python.
2. The risky code that may cause an error is placed inside the try block. If an error occurs, the except block catches the error and executes the appropriate message.
3. Example Python program:

```
try:
  num1 = 10
  num2 = 0
  result = num1 / num2
  print("Result:", result)

except ZeroDivisionError:
  print("Error: Division by zero is not allowed.")
```

4. In this program, Python tries to perform the division operation inside the try block. When division by zero occurs, the except block catches the error and prints an error message instead of stopping the program.

4. List any four built-in exceptions in Python and explain them briefly.

1. SyntaxError occurs when there is a mistake in the syntax of the Python code. This error appears when the interpreter cannot understand the structure of the program.
2. ValueError occurs when a function receives an argument of the correct type but with an inappropriate value. For example, converting a string like "abc" to an integer will cause a ValueError.
3. ZeroDivisionError occurs when a program attempts to divide a number by zero. Since division by zero is mathematically undefined, Python raises this exception.
4. IndexError occurs when a program tries to access an index that does not exist in a list or sequence. For example, accessing index 5 in a list of only three elements will cause an IndexError.

5. Explain the structure of try-except block in Python with syntax.

1. The try-except block is used in Python to handle exceptions that occur during program execution. It allows the program to manage errors and continue running without crashing.
2. The try block contains the code that may cause an exception during execution. If an error occurs in this block, Python immediately transfers control to the corresponding except block.
3. The except block handles the error by providing a suitable response such as displaying an error message. Python also provides optional blocks like else and finally to manage additional actions after the try-except process.
4. The basic syntax of try-except structure in Python is shown below:

```
try:
   # risky code
except ExceptionType:
   # code to handle the exception
else:
   # runs if no exception occurs
finally:
   # runs in all cases
```

6. Write a Python code snippet to handle multiple exceptions.

1. Multiple exceptions can occur in a Python program when different types of runtime errors happen. Python allows handling multiple exceptions using multiple **except blocks** or by grouping exceptions together.
2. Handling multiple exceptions helps the program respond differently depending on the type of error. This makes the program safer and easier to debug because each error type is handled properly.
3. Example Python code to handle multiple exceptions:

```
try:
   a = int("abc")
   b = 10 / 0
except ValueError:
   print("Invalid value entered.")
except ZeroDivisionError:
   print("Cannot divide by zero.")
```

7. What are user-defined exceptions? Explain with syntax.

1. User-defined exceptions are custom exceptions created by programmers to handle specific errors in a program. They allow developers to define their own error conditions based on application requirements.
2. In Python, user-defined exceptions are created by defining a new class that inherits from the built-in **Exception** class. This class can contain custom messages or attributes related to the error.
3. These exceptions are raised using the **raise** keyword when a particular condition occurs in the program. The exception can then be handled using the **try-except** block like any built-in exception.
4. Basic syntax for creating a user-defined exception is:

```
class MyCustomError(Exception):
  pass

raise MyCustomError("Custom error occurred")
```

8. Explain the role of the finally block in exception handling.

1. The **finally** block in Python is used to execute important code regardless of whether an exception occurs or not. It is mainly used for cleanup operations such as closing files or database connections.
2. The finally block is always executed after the try and except blocks have completed. This means it runs whether an error occurs, is handled, or does not occur at all.
3. It ensures that certain actions are performed before the program leaves the try statement. This is useful for releasing system resources and maintaining program stability.
4. For example, when working with files, the finally block can ensure that the file is properly closed. This helps prevent resource leaks and keeps the program running efficiently.

9. What are predefined cleanup actions in Python?

1. Predefined cleanup actions in Python refer to operations that are executed after the completion of a try block regardless of errors. These actions are usually defined using the **finally** clause.
2. Cleanup actions ensure that important tasks such as closing files, releasing memory, or disconnecting from databases are performed. They help maintain system resources properly even if an exception occurs.
3. The cleanup code is written inside the **finally block**, which executes under all conditions. This guarantees that the necessary cleanup tasks will always run before the program ends.
4. Therefore, predefined cleanup actions help improve program reliability and resource management. They ensure that the program leaves the try block in a safe and controlled manner.

10. Write a small Python program demonstrating try-except-finally.

1. The try-except-finally structure in Python is used to handle runtime errors and perform cleanup operations. It allows the program to handle exceptions without terminating unexpectedly.
2. The try block contains the code that might cause an error during execution. If an exception occurs, the except block catches the error and executes the corresponding code.
3. Example Python program:

```
try:
    a = 10
    b = 0
    result = a / b
    print("Result:", result)

except ZeroDivisionError:
    print("Error: Division by zero.")

finally:
    print("Execution completed.")
```

4. In this program, the division by zero causes a **ZeroDivisionError**, which is handled in the except block. The finally block executes at the end and prints a message indicating that the program execution is complete.

(5 – marks)

1. Explain different types of errors in Python with examples.

1. In Python, errors are problems that occur in a program and affect its execution. The main types of errors are **syntax errors, logical errors, and runtime errors (exceptions)**.
2. **Syntax errors** occur when the rules of Python programming are not followed. For example, missing a parenthesis or colon like print("Hello" causes a syntax error and stops the program from running.
3. **Logical errors** occur when the program runs successfully but produces incorrect results. For example, writing the wrong formula in a calculation may give the wrong output even though the program runs.
4. **Runtime errors**, also called exceptions, occur during the execution of a program. These errors happen after the syntax of the program is correct but some unexpected situation occurs.
5. Examples of runtime errors include **ZeroDivisionError, FileNotFoundError, and TypeError**. For instance, writing 10/0 will raise a ZeroDivisionError because division by zero is not allowed.

2. Write a Python program to handle multiple exceptions using multiple except blocks.

1. In Python, multiple exceptions may occur in a program depending on the type of operation performed. To manage these errors properly, Python allows the use of multiple **except blocks** after a try block.
2. Each except block handles a specific type of exception. This allows the program to display different messages or perform different actions for different errors.
3. The risky code that may cause errors is written inside the try block. If an exception occurs, Python checks the except blocks and executes the matching one.
4. Example Python program:

```
try:
    a = int(input("Enter a number: "))
    b = 10 / a
    print("Result:", b)

except ValueError:
    print("Invalid input! Please enter a number.")

except ZeroDivisionError:
    print("Division by zero is not allowed.")
```

5. In this program, a **ValueError** occurs if the user enters a non-numeric value. If the user enters zero, a **ZeroDivisionError** occurs and the corresponding except block handles the error.

3. Describe the flow of execution in try, except and finally blocks with example.

1. The try-except-finally structure controls how a program handles errors during execution. It allows the program to execute risky code safely and respond properly if an error occurs.
2. The **try block** is executed first and contains the code that might cause an exception. If no error occurs, Python skips the except block and continues executing the remaining code.
3. If an error occurs inside the try block, Python immediately stops executing that block. It then searches for the matching **except block** and executes it to handle the error.
4. After the try or except block finishes execution, the **finally block** is executed. This block always runs regardless of whether an exception occurred or not.
5. Example:

```
try:
    a = 10
    b = 0
    result = a / b
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Program execution finished")
```

6. In this example, the division operation causes a **ZeroDivisionError** which is handled in the except block. After that, the finally block runs and prints the final message indicating program completion.

4. Explain user-defined exceptions in Python with a suitable program.

1. User-defined exceptions are custom exceptions created by programmers to handle specific error conditions in a program. They allow developers to define meaningful error messages according to the requirements of the application.
2. In Python, a user-defined exception is created by defining a new class that inherits from the built-in **Exception** class. This class represents the new type of error that can occur in the program.
3. After defining the exception class, the **raise** statement is used to generate the exception when a specific condition occurs. The exception can then be handled using a try-except block.
4. User-defined exceptions help in creating clearer and more descriptive error messages. They also improve code readability and help programmers handle application-specific errors effectively.
5. Example Python program:

```
class InvalidAgeError(Exception):
  pass

try:
  age = int(input("Enter age: "))
  if age < 0:
    raise InvalidAgeError("Age cannot be negative")
  print("Age is valid")

except InvalidAgeError as e:
  print("Error:", e)
```

6. In this program, a custom exception **InvalidAgeError** is created to check invalid age values. If the age entered is negative, the exception is raised and handled in the except block with an appropriate message.

5. Write a Python program that handles file opening errors using exception handling.

1. File opening errors occur when a program tries to open a file that does not exist or cannot be accessed. In Python, such errors are handled using exception handling techniques.
2. The risky code for opening the file is written inside the **try block**. If the file cannot be opened, Python raises a **FileNotFoundError** which can be handled using the except block.
3.
```
try:
    file = open("data.txt", "r")
    content = file.read()
    print(content)
    file.close()

except FileNotFoundError:
    print("Error: File not found. Please check the file name.")
```
4. In this program, Python attempts to open a file named **data.txt**. If the file does not exist, the FileNotFoundError exception is raised and handled in the except block.
5. Exception handling prevents the program from crashing due to missing files. It also provides a clear error message so the user understands what went wrong.

6. Explain the concept of clean-up actions in exception handling.

1. Clean-up actions refer to the code that must always be executed after the completion of a try block. These actions are usually written inside the **finally block** of exception handling.
2. The finally block runs whether an exception occurs or not in the program. It ensures that important operations such as closing files or releasing resources are completed.
3. Clean-up actions are important because they help maintain proper resource management. Without cleanup operations, files or system resources might remain open and cause issues later.
4. The finally block executes before leaving the try statement under all conditions. Even if the exception is not handled by an except block, the finally block will still run.
5. Therefore, clean-up actions ensure that programs remain stable and resources are managed properly. They are commonly used for tasks like closing files, ending database connections, and freeing memory.

7. Write a program to demonstrate raising exceptions using raise keyword.

1. In Python, the **raise keyword** is used to generate an exception intentionally when a specific condition occurs. It allows programmers to control when an error should occur in a program.
2. When an exception is raised, the normal execution of the program stops and Python searches for an appropriate exception handler. If a matching except block is found, the error is handled there.
3. The raise statement is written as **raise ExceptionType("message")**. The message describes the reason for the error and helps in debugging the program.

```
try:
  age = int(input("Enter age: "))

  if age < 0:
    raise ValueError("Age cannot be negative")

  print("Age entered:", age)

except ValueError as e:
  print("Error:", e)
```

8. Write a Python program to handle invalid user input using exception handling.

1. Invalid user input occurs when a user enters data that does not match the expected format. Python uses exception handling to detect and manage such errors during program execution.
2. The user input operation is placed inside the **try block** where an exception may occur. If the user enters incorrect data, the program moves to the except block to handle the error.
3. Example Python program:

```
try:
  num = int(input("Enter a number: "))
  print("You entered:", num)

except ValueError:
  print("Invalid input! Please enter a valid integer.")
```

4. In this program, if the user enters letters or symbols instead of a number, Python raises a **ValueError**. The except block catches the error and displays a helpful message to the user.

(10 – marks)

1. Explain the complete exception handling mechanism in Python with syntax and examples.

1. Exception handling in Python is used to manage runtime errors so that the program does not crash. It allows the program to detect errors and execute alternative code to handle them properly.
2. Python provides several keywords for exception handling such as **try, except, else, and finally**. These keywords work together to detect errors, handle them, and perform necessary clean-up operations.
3. The **try block** contains the code that may cause an exception during execution. Python executes this block first and monitors it for any runtime errors.
4. If an error occurs in the try block, Python immediately stops executing that block and moves to the **except block**. The except block catches the exception and handles it by displaying a message or performing another action.
5. The **else block** is optional and runs only if no exception occurs in the try block. It is used to execute code that should run when the program works correctly.
6. The **finally block** is also optional and always executes regardless of whether an exception occurs or not. It is mainly used for cleanup actions such as closing files or releasing resources.
7. Syntax of exception handling in Python:

```
try:
   # risky code
except ExceptionType:
   # error handling code
else:
   # executes if no exception
finally:
   # cleanup code
```

8. Example:

```
try:
   a = 10
   b = 0
   result = a / b
except ZeroDivisionError:
   print("Division by zero not allowed")
finally:
   print("Program finished")
```

In this example, the exception is handled and the finally block executes at the end.

2. Describe the different types of errors and exceptions in Python with suitable examples.

1. Errors in Python are problems that occur in a program and interrupt its normal execution. They can be broadly classified into **syntax errors, logical errors, and runtime errors (exceptions)**.
2. **Syntax errors** occur when the programmer violates the rules of Python programming. These errors are detected by the interpreter before the program starts running.
3. An example of a syntax error is missing brackets or incorrect indentation in code. For example, print("Hello" will cause a syntax error because the closing parenthesis is missing.
4. **Logical errors** occur when the program runs successfully but produces incorrect results. These errors usually occur due to mistakes in the logic or formula used in the program.
5. For example, calculating the average incorrectly due to a wrong formula will produce incorrect output even though the program runs normally. Such errors are difficult to detect because the program does not stop execution.
6. **Runtime errors**, also called exceptions, occur while the program is executing. These errors happen due to unexpected conditions such as invalid input or illegal operations.
7. Examples of runtime errors include **ZeroDivisionError, TypeError, and FileNotFoundError**. For instance, the statement 10/0 raises a ZeroDivisionError because division by zero is not allowed.
8. Python provides exception handling techniques like try and except blocks to manage these runtime errors. This helps the program continue running without crashing.

3. Write a Python program that demonstrates handling multiple exceptions, finally block and user input validation.

1. In Python, a program may face different runtime errors depending on user input or program operations. Exception handling allows the program to handle these errors using multiple except blocks.
2. Multiple except blocks are used to handle different types of exceptions separately. The finally block ensures that certain code executes regardless of whether an exception occurs or not.
3. Example Python program:

```
try:
  num = int(input("Enter a number: "))
  result = 10 / num
  print("Result:", result)

except ValueError:
  print("Invalid input! Please enter a numeric value.")

except ZeroDivisionError:
  print("Error: Division by zero is not allowed.")

finally:
  print("Program execution completed.")
```

4. In this program, the user is asked to enter a number for division. If the user enters a non-numeric value, a **ValueError** occurs and is handled in the first except block.
5. If the user enters zero, Python raises a **ZeroDivisionError** and the second except block handles it. This prevents the program from crashing due to invalid mathematical operations.
6. The finally block executes at the end regardless of whether an exception occurred or not. It prints a message indicating that the program execution has finished.
7. This program demonstrates how Python can handle multiple errors while validating user input. It ensures that the program runs safely even when unexpected input is given.

4. Explain user-defined exceptions in Python and develop a program to implement them.

1. User-defined exceptions are custom exceptions created by programmers to handle specific error conditions in an application. They allow developers to define meaningful error messages and control how certain errors should be handled in a program.
2. In Python, a user-defined exception is created by defining a new class that inherits from the built-in **Exception** class. This class represents a new type of error that can occur in the program.
3. When a particular condition occurs, the **raise keyword** is used to generate the custom exception. The exception can then be caught and handled using a try-except block like any other exception.
4. User-defined exceptions improve program readability and debugging because they clearly describe the type of error. They also help handle application-specific problems separately from general system errors.
5. Example program implementing user-defined exception:

```
class InvalidAgeError(Exception):
  pass

try:
  age = int(input("Enter age: "))
  if age < 0 or age > 120:
    raise InvalidAgeError("Age must be between 0 and 120")
  print("Valid age")

except InvalidAgeError as e:
  print("Error:", e)
```

6. In this program, a custom exception **InvalidAgeError** is created to check invalid age values. If the entered age is outside the allowed range, the exception is raised and handled using the except block.

5. Illustrate the use of try-except-else-finally blocks with a detailed program.

1. In Python, the **try-except-else-finally** structure is used to handle runtime errors and manage program flow safely. It allows the program to execute risky code while providing mechanisms to handle errors and perform clean-up tasks.
2. The **try block** contains the code that may cause an exception during execution. Python first executes this block and monitors it for any runtime errors.
3. If an error occurs, Python stops the try block and executes the **except block**. This block handles the exception by displaying an error message or performing corrective actions.
4. The **else block** runs only when the try block executes successfully without any exceptions. It is useful for writing code that should run when no errors occur.
5. The **finally block** executes in all situations whether an exception occurs or not. It is mainly used for clean-up tasks like closing files or releasing system resources.
6. Example program demonstrating try-except-else-finally:

```
try:
  num = int(input("Enter a number: "))
  result = 10 / num

except ZeroDivisionError:
  print("Error: Cannot divide by zero")

except ValueError:
  print("Error: Invalid input")

else:
  print("Result:", result)

finally:
  print("Program execution completed")
```

7. In this program, errors such as invalid input or division by zero are handled using except blocks. If no error occurs the result is printed in the else block and the finally block executes at the end.

6. Discuss the concept of clean-up actions in Python exception handling and explain their importance.

1. Clean-up actions in Python refer to operations that must be executed after the try block regardless of whether an exception occurs. These actions are usually written inside the **finally block** of the exception handling structure.
2. The finally block ensures that important tasks are completed before the program leaves the try statement. This block executes in every situation whether an exception occurs, is handled, or does not occur at all.
3. Clean-up actions are commonly used for tasks such as closing files, releasing memory resources, or terminating database connections. These operations ensure that system resources are properly managed after program execution.
4. Without proper clean-up actions, resources such as open files or network connections may remain active. This can lead to resource leaks, system inefficiency, or unexpected program behavior.
5. The finally block guarantees execution even when an exception is not handled by an except block. In such cases, the clean-up code runs first and then the error is raised normally by Python.
6. Therefore, clean-up actions play an important role in maintaining program reliability and stability. They ensure that programs end safely and that important resources are released properly.

7. Write a Python program to perform division of two numbers and handle different runtime errors using exception handling.

1. In Python, runtime errors occur during the execution of a program when unexpected situations arise. These errors can be handled using exception handling so that the program does not crash.
2. When performing division of two numbers, errors such as **ZeroDivisionError** and **ValueError** may occur. These errors can be managed using multiple except blocks inside the try-except structure.
3. The division operation is placed inside the **try block** where the risky code is executed. If an error occurs, Python stops executing the try block and moves to the matching except block.
4. Example Python program:

```
try:
  num1 = int(input("Enter first number: "))
  num2 = int(input("Enter second number: "))
  result = num1 / num2
  print("Result:", result)

except ZeroDivisionError:
  print("Error: Division by zero is not allowed.")

except ValueError:
  print("Error: Please enter valid numeric values.")

finally:
  print("Program execution finished.")
```

5. In this program, if the user enters zero as the second number, a **ZeroDivisionError** occurs. If the user enters non-numeric input, a **ValueError** occurs and is handled by the corresponding except block.
6. The finally block executes at the end regardless of whether an exception occurs or not. This ensures that the program completes safely and displays the final message.

8. Explain the role of raise keyword and custom exception classes with examples.

1. In Python, the **raise keyword** is used to generate an exception manually when a specific condition occurs in a program. It allows programmers to control when an error should be triggered.
2. When the raise statement is executed, Python immediately stops the normal flow of the program. It then searches for a matching exception handler to handle the error.
3. Custom exception classes are user-defined classes that inherit from the built-in **Exception** class. They are used to represent application-specific errors with meaningful names and messages.
4. Example using raise keyword:

```
age = -5

if age < 0:
  raise ValueError("Age cannot be negative")
```

5. Example of custom exception class:

```
class InvalidMarksError(Exception):
  pass

try:
  marks = int(input("Enter marks: "))
  if marks < 0 or marks > 100:
    raise InvalidMarksError("Marks must be between 0 and 100")
except InvalidMarksError as e:
  print("Error:", e)
```

6. In this example, a custom exception **InvalidMarksError** is created to check invalid marks. When marks are outside the allowed range, the exception is raised and handled using the except block.

9. Analyze how exception handling helps in developing robust and reliable Python applications.

1. Exception handling helps Python programs manage unexpected errors during execution without crashing. This makes the application more stable and reliable in real-world situations.
2. It separates normal program logic from error-handling code using structures like try and except blocks. This separation makes the code easier to understand, maintain, and debug.
3. Exception handling improves reliability by allowing programs to continue running even when errors occur. Instead of terminating suddenly, the program can display meaningful error messages and guide the user properly.
4. It also helps developers identify the exact location and reason for errors through tracebacks. This makes debugging easier and allows programmers to fix issues more efficiently.
5. Exception handling allows specific types of errors to be handled separately using multiple except blocks. This provides better control over how different errors are managed in an application.
6. Clean-up actions using the finally block ensure that important tasks like closing files or releasing resources are always completed. This prevents resource leaks and keeps the program functioning properly.
7. It also improves user experience by preventing abrupt program termination. Users receive clear messages explaining what went wrong and how to correct their input.
8. Therefore, exception handling plays an important role in building robust and reliable Python applications. It ensures stability, better debugging, and safe handling of unexpected situations.

10. Develop a Python program for a simple banking system that handles invalid inputs using exception handling.

1. In a banking system, users may enter invalid inputs such as non-numeric values or negative amounts. Exception handling helps detect these errors and ensures the program continues running safely.
2. The program can use try and except blocks to validate user input and prevent runtime errors. This ensures that incorrect input does not cause the program to crash.
3.
```
balance = 1000

try:
  amount = float(input("Enter amount to withdraw: "))

  if amount < 0:
    raise ValueError("Amount cannot be negative")

  if amount > balance:
    raise Exception("Insufficient balance")

  balance -= amount
  print("Withdrawal successful")
  print("Remaining balance:", balance)

except ValueError:
  print("Invalid input! Please enter a valid number.")

except Exception as e:
  print("Error:", e)

finally:
  print("Transaction process completed.")
```
4. In this program, the user enters the withdrawal amount and the program checks for errors. If the user enters a negative value or non-numeric input, the program raises and handles the appropriate exception.
5. The program also checks if the withdrawal amount is greater than the available balance. If so, it raises an exception indicating insufficient funds.
6. The except blocks handle different errors and display appropriate messages to the user. This prevents the program from terminating unexpectedly.
7. The finally block executes at the end of the program regardless of whether an error occurs. It ensures that the transaction process is properly completed.